Link Chat: A Reproducible Protocol Core for Endpoint-Local State Evolution

Link Chat Research Project

September 6, 2026

Abstract

This paper presents Link Chat, a small two-party protocol core based on the Link Chat v1.0 research specification [1], designed to study how standard cryptographic mechanisms interact with endpoint-local state evolution. The construction is intentionally not introduced as a new cipher or as a complete messaging service. Its central design combines independent receiver secret state at each endpoint, strictly alternating application turns, authenticated public receiver packages, and an atomic package rotation performed only after an authenticated application input has been fully validated. Transport, mailbox, persistence, and application components are separated from the authority to advance the application state.

The paper gives a complete protocol description, a canonical byte-level encoding, a fixed 4096-byte envelope format, typed primitive interfaces, endpoint algorithms, failure rules, storage commit semantics, and reproducible test procedures. It also formalizes a complete adaptive adversary view containing protocol outputs and metadata such as timing, logical length, envelope length, acknowledgements, errors, and state-update counts. We state the structural properties checked by the associated Lean development and formulate a conditional computational theorem for a supplied concrete probability model, PPT adversary witness, wrapper-game certificates, and hybrid reduction bound. The theorem is deliberately conditional: the Lean development does not claim to derive the computational security of X25519, ML-KEM-768, HKDF, ChaCha20-Poly1305, Ed25519, or an operating-system CSPRNG from their implementations.

Keywords: protocol state machines, receiver-state rotation, non-interference, post-compromise reasoning, game-based security, Lean, reproducible wire formats.

Contents

1 Introduction

Modern end-to-end messaging systems combine cryptographic primitives with stateful protocol logic. A protocol can use well-studied primitives while still leaving important questions unanswered about how malformed, replayed, delayed, or adversarially selected inputs affect honest local state. The protocol specification isolates this question.

The core design question is the following:

> Can a peer-controlled application input influence authenticated message processing without acquiring authority to choose the honest endpoint's next private receiver state?

The answer pursued by Link Chat is architectural rather than algorithmic. Each endpoint owns only its local receiver secret state. The peer receives a public projection of that state, consisting of public keys, a key identifier, a mailbox token, a generation value, expiration information, a chain link, and an identity-bound authentication value. The application protocol advances in a strict alternating order. A valid receive operation does not mutate the current state in place: it constructs a complete candidate state, persists it through an explicit prepare/commit boundary, and exposes the application plaintext only after successful commit.

The result is a small protocol core with four intended uses:

1. a precise object and state-transition specification for independent implementations;
2. a test target for a typed Rust reference kernel;
3. a structural formalization target for Lean;
4. a composition boundary at which external primitive security assumptions can be attached without being hidden inside protocol lemmas.

The protocol is intentionally narrower than a complete production messaging system. It does not specify federation, servers, DHT placement, relay topology, account management, GUI behavior, or global traffic-analysis resistance. These omissions are not accidental: they keep the application state authority and the cryptographic research question auditable.

1.1 Contributions

This paper makes the following contributions.

1. It defines a reproducible protocol core with typed state ownership, strict turn semantics, authenticated public receiver packages, and atomic package rotation.
2. It specifies a canonical wire format with fixed field order, byte lengths, object tags, associated-data construction, and a 4096-byte envelope.
3. It gives a complete adversary model in which one endpoint may be continuously controlled while the other endpoint's private state and fresh randomness remain outside the adversary's direct input interface.
4. It separates structural theorems from computational assumptions and defines the exact correspondence required between a generic PPT adversary and an adaptive protocol-game adversary.
5. It provides a conditional reduction shape that makes every primitive certificate identify the same security parameter, concrete wrapper game, probability model, and PPT witness.
6. It defines reproducibility artifacts: a deterministic test randomness schedule, machine-readable vectors, a reference implementation, and an end-to-end fixture.

1.2 Claim boundary

The principal claim of this paper is not that the protocol is unconditionally secure in all implementations. The claim is instead layered:

1. the protocol state machine and wire contract are explicit;
2. the stated structural properties are checked under the premises of the Lean model;
3. a supplied concrete game instance can be related to named primitive wrapper games by an explicit hybrid bound;
4. computational security follows only when the external primitive certificates and implementation assumptions required by that bound are supplied.

In particular, the formalization does not internally prove the computational security of the external X25519, ML-KEM-768, HKDF, ChaCha20-Poly1305, Ed25519, or OS CSPRNG implementations. This distinction is part of the protocol's trust model.

2 Background and Position

Link Chat uses standard cryptographic interfaces rather than proposing a new primitive. The classical component is an X25519 wrapper with an ephemeral public value as its wire ciphertext [2]. The post-quantum component is ML-KEM-768, used through the standard KEM interface defined by FIPS 203 [3]. Key derivation uses HKDF-SHA-256 [4]; authenticated encryption uses ChaCha20-Poly1305 [5]; long-term identity binding uses Ed25519 [6].

The protocol should therefore be read as a state-management and composition study. Its novelty claim is deliberately limited to the arrangement of state authority, turn ownership, package publication, and proof boundaries. It does not claim a new hardness assumption, a new KEM, a new AEAD, or a general replacement for mature messaging protocols.

3 System Model and Notation

3.1 Participants and sessions

There are two endpoints, denoted Alice and Bob. A session has a fresh identifier sid, a protocol version v, and a cipher-suite identifier suite. The v1.0 suite is:

```
V1HybridMlKem768 = Ed25519 + X25519 wrapper + ML-KEM-768
                   + HKDF-SHA-256 + SHA-256 + ChaCha20-Poly1305
```

Listing 1: The frozen v1.0 cipher suite

Each endpoint has a long-term Ed25519 identity key pair and an independent local receiver secret package. The endpoints never exchange their receiver private keys.

3.2 Security parameter and probability model

Let λ be the security parameter. A concrete game instance must provide a sample space Ω_λ, all random variables used by the experiment, a probability measure over Ω_λ, an adversary strategy, resource bounds, a success predicate, and an advantage definition. In the finite Lean model, a typical sample space is a finite uniform product:

$$\Omega_\lambda = \prod_{i=1}^{m(\lambda)} \{0,1\}^{\ell_i(\lambda)},$$

where each coordinate is consumed by a declared random-source event. This finite model is an explicit semantics for the checked instance; it is not automatically an OS-randomness model.

For a bit-valued experiment $G^b_\lambda(A)$, define

$$\Pr[G^b_\lambda(A) = 1]$$

as the probability over the specified sample space and experiment coins. The distinguishing advantage is

$$\mathsf{Adv}_G(A,\lambda) = \left|\Pr[G^0_\lambda(A) = 1] - \Pr[G^1_\lambda(A) = 1]\right|.$$

3.3 PPT adversaries and witnesses

The formal endpoint used for the audited theorem is a supplied adversary witness A_λ together with a certificate that it is probabilistic polynomial time. The audited theorem is intentionally an instance theorem:

$$\forall \lambda \in \mathbb{N}, \quad \text{given } A_\lambda \text{ and its PPT witness, the stated bound holds.}$$

It is not silently upgraded to a universal theorem over every possible external PPT implementation. Such an upgrade would require a separate semantic universe of adversary programs and a proof that every program is represented by the protocol-game interface.

4 Protocol Architecture

4.1 Layering

The protocol is divided into four layers:

$$\text{Identity/Session} \to \text{Receiver State} \to \text{KEM/KDF/Message} \to \text{Transport/Mailbox/Path}.$$

Each layer exposes an interface to the next layer. A lower-layer event cannot silently authorize an unrelated application state transition.

4.2 Endpoint state

For endpoint $X \in \{A, B\}$, the production state is:

$$\begin{aligned}\mathsf{State}_X = \{&\mathsf{SessionContext}, \mathsf{IK}_{X,\mathrm{sk}}, \mathsf{IK}_{\mathrm{peer},\mathrm{pk}},\\ &\mathsf{Package}_X^{\mathrm{priv}}, \mathsf{Package}_{\mathrm{peer}}^{\mathrm{pub}}, \mathsf{expectedTurn},\\ &\mathsf{consumedMessageIDs}, \mathsf{packageChainCheckpoint}\}.\end{aligned}$$

The private package contains the local X25519 and ML-KEM private material. Its public projection contains no private key, message key, path secret, or CSPRNG state. The public projection is the only package representation that can be serialized, logged, exported, or passed to mailbox code.

4.3 State authority

The application message can carry authenticated encrypted data and a signed public return package. It can request one valid state-machine transition. It cannot be interpreted as:

```
SET_RECEIVER_PRIVATE_KEY
SET_RECEIVER_TOKEN
SET_RECEIVER_GENERATION
ROLLBACK_RECEIVER_PACKAGE
GRANT_STATE_OWNERSHIP
```

Listing 2: Forbidden interpretations of an application message

Only the protocol transition and validated storage recovery may create a new private receiver package. ACK, SACK, retry, mailbox replication, route failure, and ordinary filesystem events have no application-state authority.

5 Identity and Session Initialization

5.1 Out-of-band binding

The initial procedure must authenticate at least:

$$(v, \mathsf{suite}, \mathsf{IK}_{A,\mathrm{pk}}, \mathsf{IK}_{B,\mathrm{pk}}, \mathsf{sid}, S_0, \mathsf{Package}_A^{\mathrm{pub}}, \mathsf{Package}_B^{\mathrm{pub}}, \mathsf{Root}_A, \mathsf{Root}_B).$$

Identity fingerprints must be confirmed through a physical or equivalent authenticated channel. A network-delivered first identity claim is not sufficient.

The canonical identity-binding input is:

$$\mathsf{bind} = \mathsf{Canon}(\texttt{LinkChat/identity-binding/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{IK}_{A,\mathrm{pk}}, \mathsf{IK}_{B,\mathrm{pk}}).$$

Both endpoints sign the same binding under their own identity keys. An identity change invalidates the session and requires a new session identifier, bootstrap secret, receiver state, package chain, and mailbox-token set.

5.2 Bootstrap

Let S_0 be the out-of-band bootstrap secret. Define:

$$\begin{aligned}\mathsf{PRK}_{\mathrm{boot}} &= \mathsf{HKDF\text{-}Extract}(0^{256}, S_0),\\ K_{\mathrm{boot}} &= \mathsf{HKDF\text{-}Expand}(\mathsf{PRK}_{\mathrm{boot}}, \mathsf{Canon}(\texttt{LinkChat/bootstrap/v1}, v, \mathsf{suite}, \mathsf{sid}), 32).\end{aligned}$$

The bootstrap values are not the direct root of later receiver private states. After bootstrap confirmation, S_0, $\mathsf{PRK}_{\mathrm{boot}}$, and K_{boot} enter the implementation's erase procedure.

5.3 Initial receiver packages

Each endpoint independently samples:

$$\begin{aligned}(xsk_X, xpk_X) &\xleftarrow{\$} \mathsf{X25519.KeyGen}(\mathsf{CSPRNG}),\\ (psk_X, ppk_X) &\xleftarrow{\$} \mathsf{MLKEM768.KeyGen}(\mathsf{CSPRNG}),\\ \mathsf{Token}_X &\xleftarrow{\$} \{0,1\}^{256}.\end{aligned}$$

The initial public package also contains a locally unique KeyID, expiration information, the package-chain root predecessor, and an Ed25519 authentication value.

For owner X, the chain root is:

$$\mathsf{Root}_X = \mathsf{SHA256}(\mathsf{Canon}(\texttt{LinkChat/package-chain-root/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{IK}_{A,\mathrm{pk}}, \mathsf{IK}_{B,\mathrm{pk}}, X)).$$

6 Cryptographic Construction

6.1 Primitive profile

Table 1: Frozen v1.0 primitive profile.

Function	Primitive and parameters
Identity signature	Ed25519, 32-byte public key, 64-byte signature
Classical wrapper	X25519 ephemeral DH, 32-byte wire ciphertext
Post-quantum KEM	ML-KEM-768, 1184-byte public key, 1088-byte ciphertext
KDF	HKDF-SHA-256, 32-byte derived keys
Hash	SHA-256
AEAD	ChaCha20-Poly1305, 12-byte nonce, 16-byte tag
Freshness	OS CSPRNG or an equivalent vetted source

6.2 X25519 wrapper

The X25519 wrapper represents an ephemeral public value as its wire ciphertext:

$$(esk_X, epk_X) \xleftarrow{\$} \mathsf{X25519.KeyGen}(\mathsf{CSPRNG}), \qquad ct_X = epk_X, \qquad ss_X = \mathsf{X25519}(esk_X, pk_{X,\mathrm{recv}}).$$

The receiver computes the same shared value using its private scalar and ct_X. Low-order inputs and an all-zero shared result are rejected. The wrapper's exact wire ciphertext length is 32 bytes.

6.3 ML-KEM-768

The post-quantum component uses:

$$(pk_{PQ}, sk_{PQ}) \xleftarrow{\$} \mathsf{MLKEM768.KeyGen}(\mathsf{CSPRNG}),$$
$$(ct_{PQ}, ss_{PQ}) \xleftarrow{\$} \mathsf{MLKEM768.Encapsulate}(pk_{PQ,\mathrm{recv}}).$$

The receiver decapsulates ct_{PQ} with its local private key. Invalid ciphertexts are mapped to a stable cryptographic failure category and must not expose backend-specific failure details through the remote interface.

6.4 Hybrid derivation

The two shared secrets are combined only after domain-separated framing:

$$\mathsf{HybridInput} = \mathsf{Canon}(\texttt{LinkChat/hybrid/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{turn}, \mathsf{receiverKeyID}, ct_X, ct_{PQ}, ss_X, ss_{PQ}).$$

Then:

$$\mathsf{PRK}_{\mathrm{hybrid}} = \mathsf{HKDF\text{-}Extract}(0^{256}, \mathsf{HybridInput}).$$

Neither KEM shared secret is used directly as an AEAD key.

6.5 Message and header keys

Define:

$$\begin{aligned}
\mathsf{MessageInfo} &= \mathsf{Canon}(\texttt{LinkChat/message/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{turn}, \mathsf{direction}, \mathsf{messageID}, \mathsf{receiverKeyID}),\\
\mathsf{HeaderInfo} &= \mathsf{Canon}(\texttt{LinkChat/header/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{turn}, \mathsf{direction}, \mathsf{receiverKeyID}),\\
MK &= \mathsf{HKDF\text{-}Expand}(\mathsf{PRK}_{\mathrm{hybrid}}, \mathsf{MessageInfo}, 32),\\
HK &= \mathsf{HKDF\text{-}Expand}(\mathsf{PRK}_{\mathrm{hybrid}}, \mathsf{HeaderInfo}, 32).
\end{aligned}$$

The two keys are distinct by domain separation. A Message Key is used for one application message only.

6.6 Nonces

The message nonce is derived from a domain-separated nonce input:

$$\mathsf{NonceInfo} = \mathsf{Canon}(\texttt{LinkChat/nonce/v1}, \mathsf{sid}, \mathsf{turn}, \mathsf{direction}, \mathsf{messageID}, \mathsf{receiverKeyID}),$$
$$n_M = \mathsf{HKDF\text{-}Expand}(\mathsf{HKDF\text{-}Extract}(0^{256}, MK), \mathsf{NonceInfo}, 12).$$

Header nonce derivation uses the same context shape under an independent header-nonce domain. Nonce uniqueness depends on correct Turn, Direction, MessageID, KeyID, and Message Key binding.

6.7 Package hash and authentication

Let PackageBody be the canonical package without packageAuth. Then:

$$\begin{aligned}
\mathsf{packageHash} &= \mathsf{SHA256}(\mathsf{Canon}(\texttt{LinkChat/package-hash/v1}, \mathsf{PackageBody})),\\
\mathsf{packageAuth} &= \mathsf{Ed25519.Sign}(IK_{X,\mathrm{sk}},\\
&\qquad \mathsf{Canon}(\texttt{LinkChat/receiver-package-auth/v1}, \mathsf{PackageBody})).
\end{aligned}$$

The verifier recomputes the hash, verifies the signature under the pinned identity key, and checks the chain predecessor.

6.8 AEAD and associated data

The logical Header is sealed as:

$$C_H = \mathsf{Seal}(HK, n_H, \mathsf{HeaderAD}, \mathsf{Canon}(\mathsf{Header})),$$

and the fixed PayloadFrame is sealed as:

$$C_M = \mathsf{Seal}(MK, n_M, \mathsf{MessageAD}, \mathsf{PayloadFrame}).$$

The HeaderAD is:

$$\mathsf{HeaderAD} = \mathsf{Canon}(\texttt{LinkChat/header-ad/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{turn}, \mathsf{direction}, \mathsf{receiverKeyID}).$$

The encrypted Header contains MessageID, so MessageID cannot appear in HeaderAD. After successful Header open and canonical decoding, the receiver constructs:

$$\mathsf{MessageAD} = \mathsf{Canon}(\texttt{LinkChat/message-ad/v1}, v, \mathsf{suite}, \mathsf{sid}, \mathsf{turn}, \mathsf{direction}, \mathsf{messageID}, \mathsf{receiverKeyID}).$$

AEAD open is a single decrypt-and-authenticate operation. Failure stops processing and cannot update consumed IDs or generate a new package.

7 Canonical Wire Format

7.1 Object encoding

Every canonical object begins with a nine-byte header:

Offset	Size	Meaning
0	1	Object tag
1	2	Canonical format version, unsigned big-endian
3	2	Protocol major version, unsigned big-endian
5	2	Protocol minor version, unsigned big-endian
7	2	Field count, unsigned big-endian

Every field is encoded as $\mathsf{u32}_{\mathrm{be}}(\mathsf{length})$ followed by the field bytes. Fields are neither omitted nor reordered. Decoders reject truncation, overflow, unsupported tags or versions, invalid field counts, semantic length errors, trailing bytes, and non-canonical re-encoding.

7.2 Object registry

7.3 ReceiverPackage

The public package has eleven fields:

Table 2: v1.0 object tags and field counts.

Tag	Object	Fields after object header
0x01	ApplicationMessage	8
0x02	ReceiverPackage	11
0x03	Envelope	6
0x04	ACK	6
0x05	SACK	6
0x06	MailboxRecord	2
0x07	Header	10
0x7f	Context	variable

Index	Field	Encoding
0	cipher suite	u16
1	session ID	u64
2	turn	u64
3	generation	u64
4	key ID	u64
5	X25519 public key	32 bytes
6	ML-KEM-768 public key	1184 bytes
7	mailbox token	32 bytes
8	expiration	u64
9	previous package hash	32 bytes
10	package authentication	64 bytes

Its canonical encoded length is exactly 1439 bytes. Private X25519 and ML-KEM values are not fields of this object.

7.4 Header and Envelope

The ten Header fields are:

$$(\mathsf{suite}, \mathsf{sid}, \mathsf{turn}, \mathsf{direction}, \mathsf{messageID},\\ \mathsf{receiverKeyID}, \mathsf{messageType}, \mathsf{senderPackageHash},\\ \mathsf{receiverPackageHash}, \mathsf{returnReceiverPackage}).$$

The canonical logical Header is 1588 bytes. Header AEAD adds a 16-byte tag, giving an encrypted Header field of 1604 bytes.

The six Envelope fields are:

$$(\mathsf{mailboxToken}, ct_X, ct_{PQ}, C_H, C_M, \mathsf{padding}).$$

Their fixed offsets are:

Field	Offset range	Value length
Object header	0–8	9
Mailbox token	9–44	32
X25519 ciphertext	45–80	32
ML-KEM ciphertext	81–1172	1088
Encrypted Header	1173–2780	1604
Message ciphertext	2781–4091	1307
Padding	4092–4095	0

The complete canonical Envelope is exactly 4096 bytes.

7.5 PayloadFrame

For application bytes m of length $\ell \leq 1289$:

$$\mathsf{PayloadFrame}(m) = \mathsf{u16}_{\mathrm{be}}(\ell) \,||\, m \,||\, 0^{1289-\ell}.$$

The frame is exactly 1291 bytes. ChaCha20-Poly1305 adds a 16-byte tag, producing a 1307-byte message ciphertext.

7.6 ACK, SACK, and MailboxRecord

ACK and SACK are transport/control objects. Their protocol version is carried by the common object header. ACK has payload fields (suite, sid, turn, direction, messageID, deliveryStatus). SACK has payload fields (suite, sid, turn, direction, baseMessageID, bitmap). A MailboxRecord binds one opaque token to one canonical Envelope.

None of these control objects can advance a turn, rotate a receiver package, mark an application message consumed, or restore an erased message key.

8 Endpoint Algorithms

8.1 Send preparation

On an expected turn t, the endpoint first checks that it is the leader:

$$\mathsf{Leader}(t) = \begin{cases} \mathsf{Alice}, & t \bmod 2 = 0, \\ \mathsf{Bob}, & t \bmod 2 = 1. \end{cases}$$

The send procedure then selects the current verified peer package, samples ephemeral X25519 material, encapsulates under both receiver public keys, derives the hybrid keys, constructs the Header and PayloadFrame, seals both, and returns an Envelope plus a complete pending state. Preparation alone does not publish state.

```
prepare_send(plaintext):
    require local_endpoint == Leader(expected_turn)
    require peer_package is current, verified, unexpired
    sample ephemeral X25519 key
    encapsulate to peer X25519 and ML-KEM public keys
    derive PRK_hybrid, HK, MK, header_nonce, message_nonce
    construct Header with current sender package
```

```
    construct fixed PayloadFrame(plaintext)
    seal Header with HeaderAD
    seal PayloadFrame with MessageAD
    construct 4096-byte Envelope
    return PreparedSend(envelope, candidate_state)
```

Listing 3: Abstract send preparation

8.2 Receive processing

The receive order is normative:

```
bounded decode of the 4096-byte Envelope
outer Token and context checks
X25519 and ML-KEM decapsulation
Hybrid KDF
Header AEAD Open with HeaderAD
canonical Header decode
Session, Turn, Direction, MessageID, and KeyID checks
receiver package hash check
return Package signature and chain checks
Message KDF
Message AEAD Open with MessageAD
PayloadFrame length and zero-padding checks
duplicate and consumed-message checks
fresh local Receiver Package generation
complete pending-state construction
durable prepare and atomic commit
application delivery
```

Listing 4: Normative receive processing order

The order prevents an unauthenticated field from becoming a state-authority input. In particular, the receiver does not generate and persist a new package until the entire message, authenticated return package, and payload have passed validation.

8.3 Successful receive transition

After all checks succeed, the receiver samples a new local package:

$$
\begin{aligned}
(xsk', xpk') &\xleftarrow{\$} \mathsf{X25519.KeyGen}(\mathsf{CSPRNG}),\\
(psk', ppk') &\xleftarrow{\$} \mathsf{MLKEM768.KeyGen}(\mathsf{CSPRNG}),\\
\mathsf{Token}' &\xleftarrow{\$} \{0,1\}^{256},\\
\mathsf{generation}' &= \mathsf{generation} + 1,\\
\mathsf{previousHash}' &= \mathsf{Hash}(\mathsf{currentPublicPackage}).
\end{aligned}
$$

The new private material, public projection, expected turn, peer package, and consumed MessageID set are committed as one logical state.

8.4 Failure behavior

Every following condition is a no-state-change result:

- malformed, non-canonical, truncated, oversized, or unsupported input;
- session, token, turn, direction, key-ID, expiration, or context mismatch;
- replay, future turn, duplicate message, package-hash, signature, or chain failure;
- X25519, ML-KEM, Header AEAD, Message AEAD, or PayloadFrame failure;
- storage preparation, durability, or commit failure.

No failure changes the current package, peer package, expected turn, consumed IDs, generation, or retained message-key state. A rejected plaintext is not delivered.

9 Storage and Recovery

9.1 Prepare/commit contract

The abstract storage interface is:

$$\mathsf{load}() \to \mathsf{CurrentState}, \quad \mathsf{prepare\_commit}(p) \to \mathsf{PreparedCommit},$$
$$\mathsf{commit}(c) \to \mathsf{Result}, \quad \mathsf{recover}() \to \mathsf{CurrentState}.$$

A recommended durable order is:

1. write and validate Pending;
2. complete a durability barrier;
3. write the commit marker;
4. complete a durability barrier;
5. atomically publish Current;
6. complete the required directory or medium barrier;
7. retire the old state;
8. attempt implementation-defined zeroization.

The old private state must not be erased before the new state is durable.

9.2 Recovery matrix

Atomic rename, filesystem synchronization, or ordinary deletion does not by itself prove rollback resistance or physical erasure. A true rollback-resistance claim requires an external monotonic property, such as an appropriate hardware-backed or service-backed counter.

Table 3: Required recovery behavior.

Crash or corruption condition	Required result
Pending incomplete	Discard Pending; retain Current
Pending complete without marker	Discard Pending; retain Current
Marker present without published Current	Adopt Pending as Current
Current published while old state remains	Use new Current; retire old later
Record hash mismatch	Refuse normal recovery and report storage corruption
Package-chain mismatch	Refuse normal recovery and report rollback or fork
Private/public projection mismatch	Refuse recovery

10 Security Games

10.1 Complete adversary view

In the Alice-compromised experiment, the adversary may choose or observe Alice's local state, plaintexts, randomness, ciphertexts, tokens, application actions, network actions, transport actions, mailbox and DHT actions, and replay, delay, drop, modification, and injection behavior. It also receives Bob's declared public outputs and protocol-level observations.

The complete view is modeled as:

$$\begin{aligned}\mathsf{View}_A = (&\mathsf{controlledState}, \mathsf{plaintexts}, \mathsf{randomness}, \mathsf{ciphertexts}, \mathsf{tokens},\\ &\mathsf{networkTrace}, \mathsf{transportTrace}, \mathsf{mailboxTrace}, \mathsf{DHTTrace},\\ &\mathsf{peerPublicOutputs}, \mathsf{ACKError}, \mathsf{acceptReject}, \mathsf{eventTime},\\ &\mathsf{logicalLength}, \mathsf{envelopeLength}, \mathsf{stateUpdateCount}, \mathsf{packageUpdateCount}).\end{aligned}$$

If an experiment exposes fewer components, it is a projection rather than the complete view.

10.2 Structural state non-interference

Let s be an honest state, ρ a fixed local fresh schedule, and x_1, x_2 peer inputs. The strong structural property is:

Definition 10.1 (State non-interference)**.** *If both inputs satisfy the same acceptance branch, then*

$$\mathsf{nextSecret}(s, x_1, \rho) = \mathsf{nextSecret}(s, x_2, \rho).$$

The property is stronger than merely stating that peer bytes do not appear syntactically in a state field. It says that, for a fixed local fresh schedule, peer choice cannot choose the honest endpoint's next private secret.

10.3 Future-state challenge

The Real world uses the honest endpoint's generated future receiver state. The Random world uses an independent random future-state value. Both worlds use the same initial state, adaptive strategy, trace schedule, timing, length metadata, ACK/error interface, accept/reject interface, and challenge message inputs. Only the challenge material derived from the future state may differ.

10.4 Future-message challenge

For two equal-length messages M_0, M_1, sample $b \xleftarrow{\$} \{0,1\}$ and return:

$$C^* = \mathsf{Seal}(K_b, n, AD, \mathsf{PayloadFrame}(M_b)).$$

The challenge is a real-format ciphertext inside a fixed-size envelope, not an exposed internal state value. The game therefore tests message indistinguishability through the same observable interface used by the protocol.

10.5 FS, PCS, authentication, and KCI

Forward secrecy gives the adversary a current state and challenges it on a previously erased message key or message. Post-compromise security gives the adversary state at time t_0, restores honest behavior at t_1, requires a fresh local package, and challenges a message after recovery. Authentication challenges package forgery across sessions, turns, KeyIDs, or package chains. KCI is split into the two directions of identity-key compromise; an affected session must be invalidated and re-established through identity binding.

11 Structural Properties and Formalization

The associated Lean project is organized into protocol definitions, structural games, standard-crypto interfaces, parameterized probability semantics, concrete reduction bookkeeping, implementation semantics, and trust-boundary audits. The relevant conceptual modules include:

```
LinkChat.lean
SecurityGames.lean
RefinedProtocol.lean
StandardCrypto.lean
ComputationalGames.lean
ParameterizedConcreteSecurity.lean
ConcreteSECCReduction.lean
ConcreteSecurityReductions.lean
ImplementationSemantics.lean
TrustBoundaryAudit.lean
AuditHardening.lean
```

Listing 5: Conceptual organization of the Lean development

Under their explicit premises, the formalized structural layer covers:

- strict alternation and unique leader selection;
- invalid-input no-state-change behavior;
- replay, future-turn, duplicate, and rollback handling;
- atomic receiver-package rotation;
- package binding and state-injection resistance;
- bidirectional state non-interference;
- continuous adaptive attack traces;

- abstract KEM and key-derivation correctness.

The formalization also provides interfaces for finite and parameterized probability semantics, PPT resource certificates, the complete adversary view, future-state and future-message challenges, FS/PCS/authentication/KCI games, named primitive wrapper games, hybrid advantage addition, and finite-term negligible closure.

11.1 Adversary refinement

The generic PPT adversary must be related to the protocol-game adversary rather than merely given the same name. The required refinement theorem has the form:

$$\forall h, \quad \mathsf{protocolAdversary}(A, h) = \mathsf{refine}(A, \mathsf{CompleteAdversaryView}(h)),$$

where h is the complete observable history and the refinement preserves the action, resource, and observation interfaces. This establishes that the protocol game can make adaptive decisions from the same history that the abstract PPT witness is allowed to see.

The audited theorem additionally requires that the challenge, event time, logical length, fixed envelope length, ACK/error values, accept/reject observations, state-update count, and package-update count are the same components on both sides of the binding.

12 Conditional Computational Theorem

12.1 Primitive certificates

Each certificate must identify all of the following:

1. the designated wrapper game, not merely a primitive name;
2. the same probability model used by the protocol game;
3. the same security parameter λ or index n;
4. a declared PPT adversary witness and its resource bound;
5. the simulator and interface used by the corresponding hybrid hop.

Thus, a field labelled "x25519" is not enough. A valid certificate is a statement about the exact X25519 wrapper game invoked by the concrete reduction.

12.2 Hybrid bound

For a supplied concrete protocol game instance and witnesses, the intended reduction is:

$$\begin{aligned}\mathsf{Adv}_{\mathrm{SECC}}(A, \lambda) \le\ & \mathsf{Adv}_{\mathrm{State}}(A, \lambda) + \mathsf{Adv}_{\mathrm{X25519}}(B_X, \lambda) + \mathsf{Adv}_{\mathrm{MLKEM}}(B_{PQ}, \lambda) \\ & + \mathsf{Adv}_{\mathrm{HKDF}}(B_{KDF}, \lambda) + \mathsf{Adv}_{\mathrm{AEAD,conf}}(B_C, \lambda) \\ & + \mathsf{Adv}_{\mathrm{AEAD,int}}(B_I, \lambda) + \mathsf{Adv}_{\mathrm{CSPRNG}}(B_R, \lambda). \end{aligned} \tag{1}$$

The state term is zero when Real and State-Isolated are extensionally equal under the same observable interface. Otherwise it remains an explicit term.

Theorem 12.1 (Audited concrete-instance SECC theorem)**.** *Let G^0_λ and G^1_λ be one concrete Real/Random protocol-game pair with the same complete observable interface. Let A_λ be a supplied PPT witness with an adversary-refinement theorem. Suppose the concrete reduction supplies wrapper-game certificates for X25519, ML-KEM-768, HKDF-SHA-256, ChaCha20-Poly1305 confidentiality and integrity, and the honest fresh-randomness source, all at the same parameter λ. If the hybrid hops satisfy equation (1), then:*

$$\mathsf{Adv}_{\mathrm{SECC}}(A_\lambda, \lambda) \leq \textit{the right-hand side of Equation (1).}$$

If, in addition, the state term is zero and every primitive term is negligible in λ, then:

$$\mathsf{Adv}_{\mathrm{SECC}}(A_\lambda, \lambda) \leq \mathsf{negl}(\lambda).$$

Proof. The first inequality is the triangle inequality applied to the explicitly declared game sequence. Each adjacent hybrid changes one named component and is bounded by its wrapper certificate. Adversary refinement allows the same adaptive strategy to be replayed in the simulator because the simulator receives the same complete history and exposes the same challenge and metadata. If Real and State-Isolated are extensionally equal, their event probabilities are equal and the state term is zero. Finally, a finite sum of negligible functions is negligible, so the additional premises imply the final conclusion. □

Remark 12.2. *The theorem is deliberately not a theorem that every PPT adversary, every implementation, or every external cryptographic library is secure. It is a theorem for the supplied, audited, parameterized game instance and its declared witnesses.*

12.3 What the theorem does not establish

The theorem does not by itself establish physical erasure, rollback resistance on an ordinary filesystem, resistance to memory disclosure, resistance to side channels, secure cross-process coordination, availability against a malicious mailbox, anonymity, or traffic-analysis protection. Those are separate implementation or deployment properties.

13 Reference Implementation and Reproducibility

The Rust reference kernel is divided into typed, maintainable crates:

Crate	Role
`linkchat-types`	fixed-size semantic identifiers and public types
`linkchat-protocol`	canonical objects, codecs, contexts, and wire errors
`linkchat-crypto`	typed primitive-provider boundary
`linkchat-core`	pure protocol transitions and package logic
`linkchat-engine`	prepare/commit endpoint orchestration
`linkchat-storage`	durable transaction and recovery adapters
`linkchat-transport`	packet, retry, ACK, SACK, and metadata boundary
`linkchat-mailbox`	token-bound mailbox interface and in-memory adapter
`linkchat-verification`	properties, vectors, fuzz smoke, and differential checks
`linkchat-ffi`	opaque-handle C ABI boundary
`linkchat-cli`	inspection and vector commands

For deterministic test artifacts, define:

$$R(\mathsf{seed}, i) = \mathsf{SHA256}(\mathsf{seed} \,||\, \mathsf{u64}_{\mathrm{be}}(i)).$$

This schedule is test-only. Production implementations must use an operating-system CSPRNG or an equivalent vetted source.

The published vector set includes identity binding, bootstrap, package structure, application-message encoding, context AAD, payload frame, envelope layout, HKDF framing, AEAD round trips, KEM boundaries, and replay/recovery behavior. Structural vectors use patterned bytes and exact lengths; cryptographic test material is explicitly marked test-only.

The reference kernel was checked with:

```
cargo fmt --all -- --check
cargo check --workspace
cargo test --workspace
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo run -p linkchat-cli -- test-vector
```

Listing 6: Reference verification commands

These commands provide implementation and vector evidence. They do not replace the Lean theorems or external primitive-security certificates.

13.1 Turn-zero fixture

A complete reproducible fixture uses $\mathsf{sid} = 42$, Turn 0, Alice as leader, $\mathsf{messageID} = 7$, receiver KeyID 99, expiration 1700000000, and UTF-8 payload `hello`. The PayloadFrame is:

$$\texttt{0005} \,||\, \texttt{68656c6c6f} \,||\, 0^{1284},$$

with length 1291 bytes. The resulting Envelope has length 4096 bytes and contains no plaintext message ID, plaintext payload, private key, or secret state.

14 Discussion

The protocol's main design choice is to separate input authentication from state authority. An application message can be accepted, rejected, replayed, or delivered, but it cannot directly select the next private receiver state. This produces a clean structural property that is useful before any computational assumption is applied.

The strict alternating turn rule is a deliberate restriction. It avoids ambiguity from concurrent updates, generic receive windows, and multiple valid commits for one endpoint. Transport may perform retries or parallel mailbox retrieval, but application commit is serialized. A future extension that introduces windows, arbitrary reordering, or concurrent commits must revise the state model, wire semantics, security games, and proof obligations.

The fixed envelope and payload frame make the protocol reproducible and simplify comparison of observable lengths. They do not, however, constitute a global traffic-analysis defense. The adversary model may expose logical length, event timing, ACK/error results, and state update counts precisely because the goal is to avoid hiding these channels in an informal security claim.

15 Limitations and Research Boundary

The following boundaries are explicit.

1. **Primitive security:** The paper assumes named computational security certificates for the concrete wrappers. It does not prove the low-level security of X25519, ML-KEM-768, HKDF, AEAD, Ed25519, or the OS CSPRNG inside Lean.
2. **Storage:** The protocol specifies a crash-safe transaction contract, but rollback resistance requires an external monotonic property and physical erasure remains platform-dependent.
3. **Deployment:** Servers, DHTs, relays, production mailboxes, account systems, and network availability are outside the core.
4. **Side channels:** Memory disclosure, timing leakage, cache behavior, swap, crash dumps, backups, and allocator copies are not modeled as eliminated.
5. **Adversary theorem scope:** The audited final theorem is for a supplied parameterized game instance and PPT witness with an explicit semantic refinement. It is not an automatic universal quantification over arbitrary programs.
6. **Privacy scope:** The core does not establish anonymity, token unlinkability, global traffic-analysis resistance, or mailbox availability.

These limitations are part of the research result. They prevent a structural protocol proof from being mistaken for a complete deployment security certification.

16 Conclusion

Link Chat is a reproducible protocol core, based on the Link Chat v1.0 research specification [1], for studying how authenticated application inputs interact with endpoint-local secret-state evolution. Its design uses standard cryptographic primitives and concentrates the protocol contribution in four mechanisms: independent receiver state, strict turns, authenticated public packages, and atomic local rotation after complete validation.

The resulting specification is sufficiently concrete for independent implementation: object tags, field order, offsets, lengths, cryptographic domains, failure behavior, storage boundaries, and test vectors are fixed. The associated formalization provides a structural verification layer and a conditional computational composition layer. The final computational statement is intentionally honest about its trust boundary: it applies to a declared concrete game instance with PPT witnesses and named primitive certificates, and it does not claim that Lean has proved the external libraries or a full messaging deployment.

A Wire-Length Derivations

Each field contributes four bytes of length prefix plus its value length. Thus the public ReceiverPackage length is:

$$9 + 4(11) + 2 + 8 + 8 + 8 + 8 + 32 + 1184 + 32 + 8 + 32 + 64 = 1439.$$

The logical Header length is:

$$9 + 4(10) + 2 + 8 + 8 + 1 + 8 + 8 + 1 + 32 + 32 + 1439 = 1588.$$

The Envelope length is:

$$9 + 4(6) + 32 + 32 + 1088 + 1604 + 1307 + 0 = 4096.$$

The PayloadFrame and ciphertext lengths are:

$$2 + 1289 = 1291, \qquad 1291 + 16 = 1307.$$

B Failure Categories

The stable remote-visible categories are:

Malformed	UnsupportedVersion	UnsupportedSuite	ContextMismatch
Replay	FutureTurn	Duplicate	PackageInvalid
KemFailure	AuthenticationFailure	PayloadInvalid	StorageFailure
Unavailable			

Internal diagnostics may be more detailed, but must not expose private keys, message keys, complete plaintexts, or CSPRNG state.

C Artifact Locations

The English protocol specification and vectors are maintained in the companion research-document repository:

```
LinkChatDocuments repository root
```

The Lean formalization and Rust reference kernel are maintained together in the companion implementation repository:

```
LinkChat repository root
```

References

[1] Link Chat Research Project. Link Chat Protocol Research Specification, v1.0-research. English protocol specification and reproducibility artifacts, 2026.

[2] Adam Langley, Mike Hamburg, and Sean Turner. Elliptic Curves for Security. RFC 7748, Internet Engineering Task Force, 2016.

[3] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS PUB 203, 2024.

[4] Hugo Krawczyk and Pasi Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869, Internet Engineering Task Force, 2010.

[5] Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, Internet Engineering Task Force, 2018.

[6] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, Internet Engineering Task Force, 2017.

[7] Victor Shoup. Sequences of Games: A Tool for Taming Complexity in Security Proofs. Cryptology ePrint Archive, Report 2004/332, 2004.